



Vizier: Black-Box Optimization Service

Google Deep Mind Publication
Approval Request Ref: TTCR_1265139f
Approval Status: Approved
Requester email: gpk@google.com
Request date: 2/18/2025 3:46:35 PM

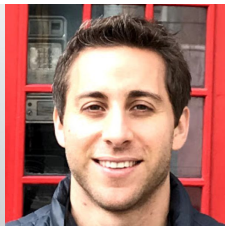
Greg Kochanski
Staff Research Engineer
gpk@google.com

2025-03-17

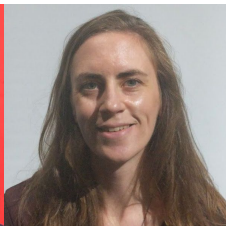
Setareh Ariaifar



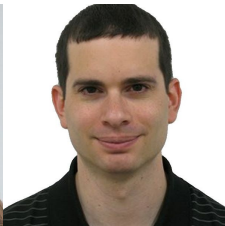
Lior Belenki



Emily Fertig



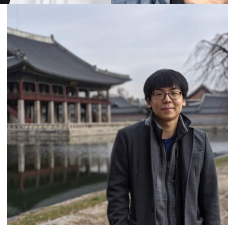
Daniel Golovin



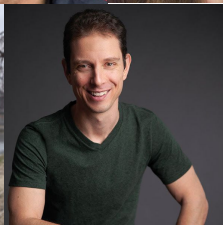
Tzu-Kuo Huang



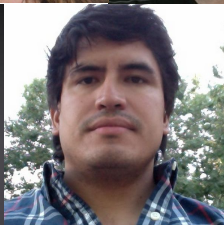
Greg Kochanski



Chansoo Lee



Sagi Perel



Adrian Reyes



Xingyou Song



Richard Zhang

Overview

What is Black Box Optimization?

What is Bayesian Optimization?

How do You Make It Work?

Gaussian Process Regression.

Details Matter.

Features of Vizier

What Is Black Box Optimization?

- You're optimizing something where you have to compute or measure the objective function.
 - You have no gradients or other inside information.
- A classic Vizier use-case:
 - You have a system with **knobs** (but not thousands of knobs)
 - You want to find the **best knob setting** to maximize an **objective** (e.g. a single float metric or multiple metrics)
 - It's **expensive** to change the knobs and measure the objective (it takes more than a few seconds to make a measurement)

Black Box Optimization Use Cases At Google

- ML hyperparameter tuning
 - Find the best hyperparameters for an ML model (e.g. BigQuery ML)
- Auto-ML
 - Find the best model type & hyperparameters for a dataset (e.g. [AutoML Tables](#))
- System optimization
 - Auto-tune systems online to conserve resources and speed up performance
- Hardware Design
 - Find [better designs faster](#), optimize chip test plans, find better quantizations
- Business Insights
 - Find the Pareto frontier for sets of metrics

Use Cases At Google: Hardware Design

- Google pixel camera white balance correction model is optimized using Vizier
- Nest Hub 2 sleep sensing was optimized with Vizier
- Apollo: Transferable Architecture Exploration
 - Generates custom hardware accelerator designs optimized for specific models (such as EfficientNet) with up to 6x better
- Vizier optimizes chip test plans

Use Cases At Google: System optimization

- Optimize dataflow pipelines: minimize runtime cost, by keeping error rate and latency below pre-specified thresholds
- Find optimal codec parameters for YouTube: better parameters yield both serving and compute savings
- Shaping of server jobs lead to savings of \$\$\$TB of RAM and \$M CPUs
- Train Google Image search to control the variety of search results.

Culinary Black Box Optimization.

Bayesian Optimization for a Better Dessert



Lessons learned



Use Cases At Google: Is Vizier Always a Win?

Not always.

- For systems which are heavily hand-optimized, Vizier may not be able to find a better solution.
 - Vizier shines when the number of parameters > 3 . Humans have a hard time visualizing high dimensional data.
- Needle in a haystack situations.
- Highly non-stationary functions (though we have some workarounds).
- Good integrations take time to build.

Overview

What is Black Box Optimization?

What is Bayesian Optimization?

How do You Make It Work?

Gaussian Process Regression.

Details Matter.

Features of Vizier

What Is Bayesian Optimization?

- You have an probabilistic model, $\mathbf{R}(x)$, of a system, $F(x)$.
 - $y = \mathbf{R}(x)$, where $\mathbf{R}$ is sampled from a probability distribution.
- Bayes' Theorem:
 - Accumulate support points (data) to constrain the model.
 - $P(\mathbf{R} \mid \text{Data}) = P(\text{Data} \mid \mathbf{R}) * P(\mathbf{R}) / \text{Sum over all } \mathbf{R} \text{ of } P(\text{Data} \mid \mathbf{R}) * P(\mathbf{R})$
 - Given enough well-distributed data, the remaining $\mathbf{R}(x)$ should describe your system accurately.
- Search $\mathbf{R}(x)$ to find an interesting & informative x -value, x^* .
- Evaluate $F(x^*)$ to expand the training dataset
 - $\text{Data} \leftarrow \text{Data} + \{x^*, F(x^*)\}$.
- Repeat until $\mathbf{R}(x)$ matches $F(x)$.

- Given: A (possibly noisy) objective function $f(x)$.
- Given: A set of training data $\{D_i\}$.
- Given: A model $\mathbf{R}(x; \{D_i\})$ that is built from the available training data.
 - The model maps from $x \rightarrow (\text{mean}, \text{standard deviation})$
- Given: An acquisition function $\mathbf{A}$ built from $\mathbf{R}$.
 - $\mathbf{A}(x; \{D_i\}) = \mathbf{A}(\mathbf{R}(x; \{D_i\}))$
 - $\mathbf{A}(\text{mean}, \text{standard deviation})$ increases with both *mean* and *standard_deviation*.
- Algorithm:
 - Until converged, do:
 - Find $x^* \leftarrow \operatorname{argmax}_x \mathbf{A}(x; D)$
 - Measure $y^* \leftarrow f(x^*)$
 - Append: $D \leftarrow D + (x^*, y^*)$

Overview

What is Black Box Optimization?

What is Bayesian Optimization?

How do You Make It Work?

Gaussian Process Regression.

Details Matter.

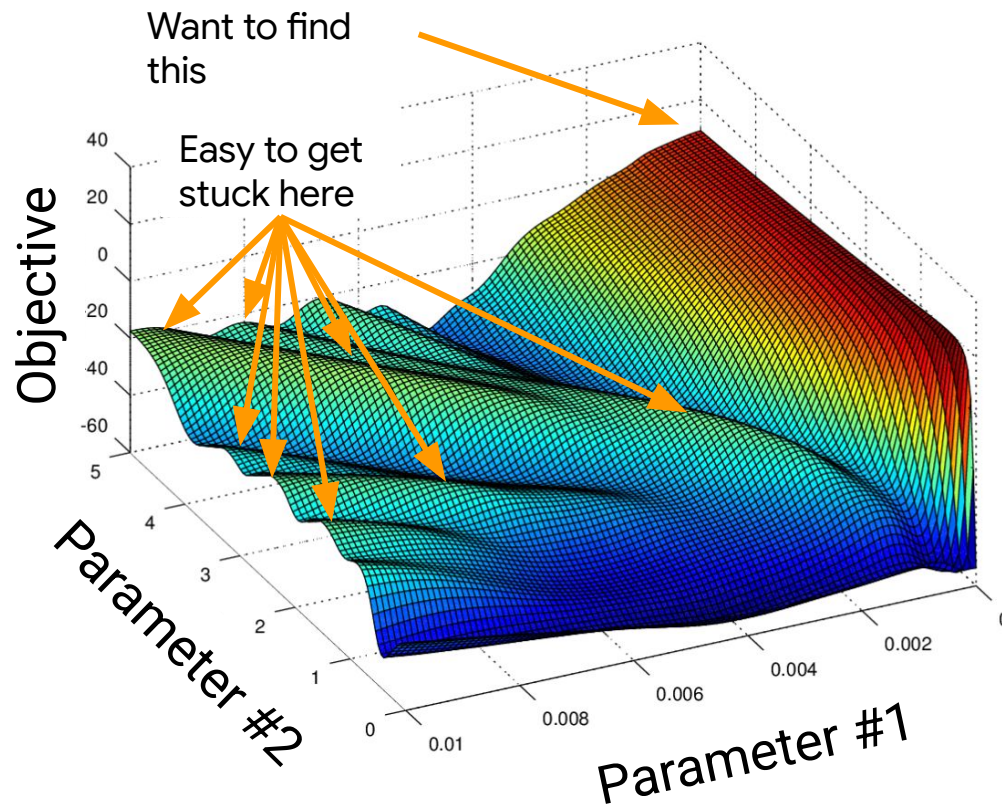
Features of Vizier

- Algorithm:
 - Until converged, do:
 - Find $x^* \leftarrow \operatorname{argmax}_x \mathbf{A}(x; D)$
 - Measure $y^* \leftarrow f(x^*)$
 - Append: $D \leftarrow D + (x^*, y^*)$

What properties must we give to $\mathbf{A}(x; D)$ so that:

1. The algorithm will converge to a single simple optimum?
2. The algorithm will eventually find the best optimum?
 - E.g. it will eventually explore the entire space.
3. The algorithm is robust against non-Gaussian y -distributions?
4. The algorithm will converge fairly quickly for most "nice" functions?

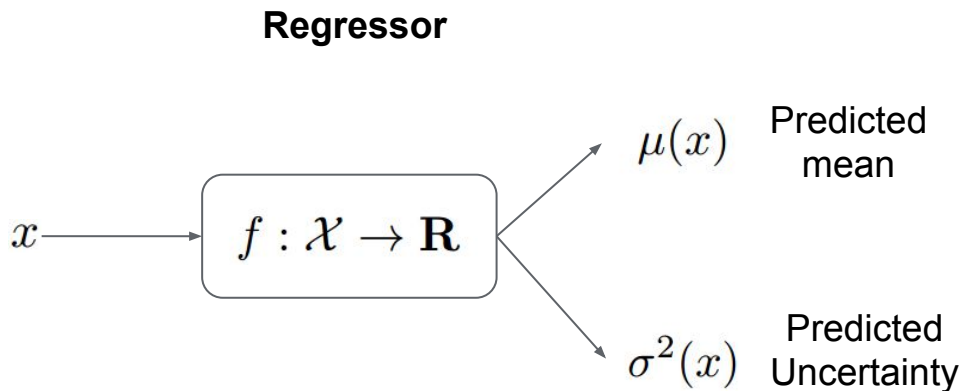
Most Good Optimizers Need to Explore.



(Large neural network models are an exception.)

Components of typical Bayesian Optimization

Proprietary / Confidential



Acquisition function

$$\alpha(x; \mathcal{D}_n) = \mu(x) + \kappa\sigma(x)$$

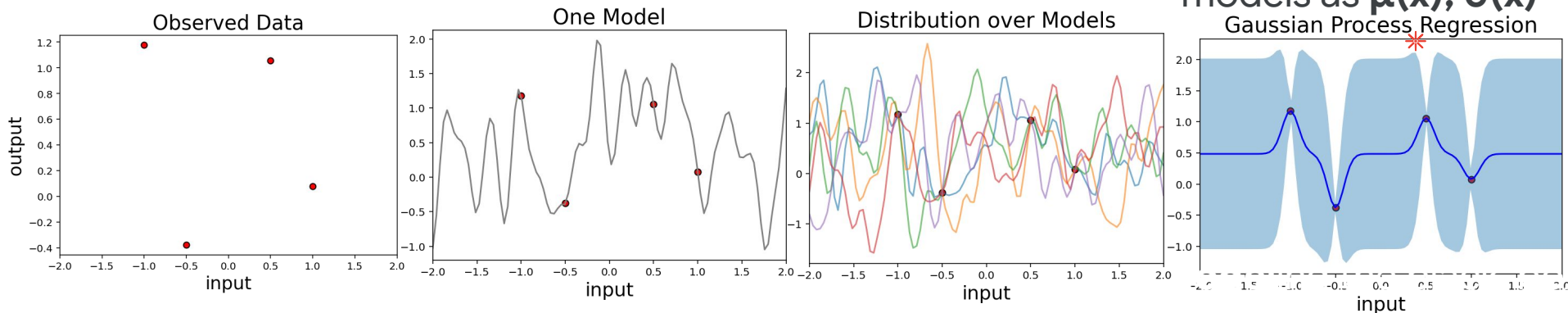
... or
Expected Improvement,
or similar functions.

Bayesian optimization in a



Represent
distribution over

models as $\mu(x)$, $\sigma(x)$
Gaussian Process Regression

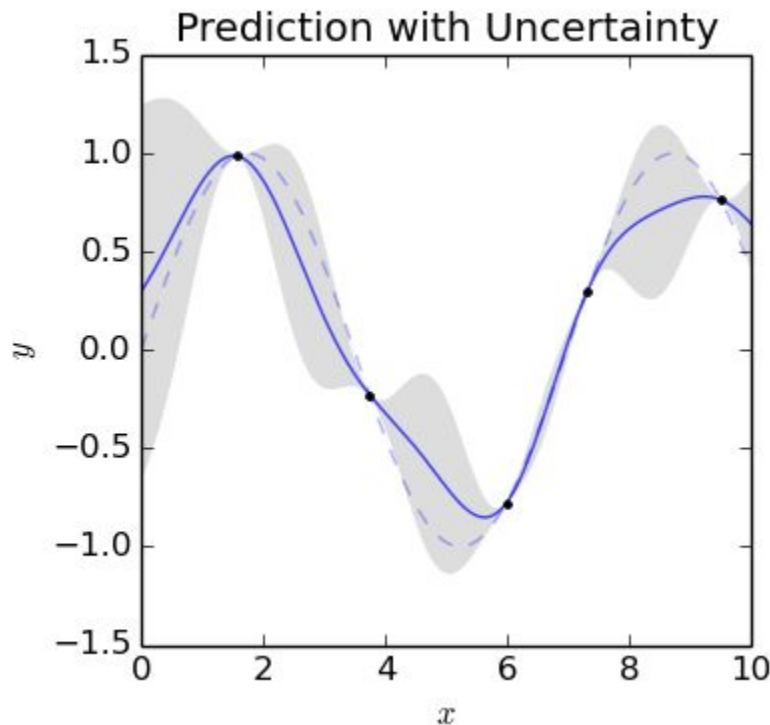


Find argmax of **UCB** :

○ $\mu(x) + b * \sigma(x)$.

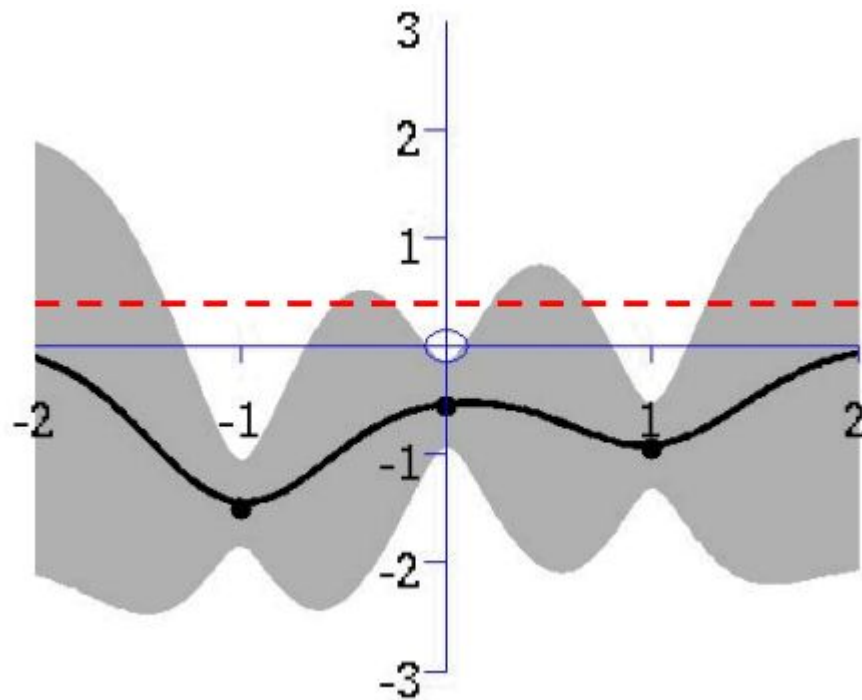
Over-exploring: searching where it's already known to be good.

- Recall $\mathbf{A}(\mathbf{x}) = \mu(\mathbf{x}) + \mathbf{b} * \sigma(\mathbf{x})$
- Typically, $\sigma(\mathbf{x})$ increases away from support points.
- Typically, $\mu(\mathbf{x})$ curves downward near the best support point.
- If $\mathbf{A}(\mathbf{x})$ curves downward around the best known observation, you can get trapped in the vicinity.
- So, if $\mathbf{b} \rightarrow 0$, you can get trapped.



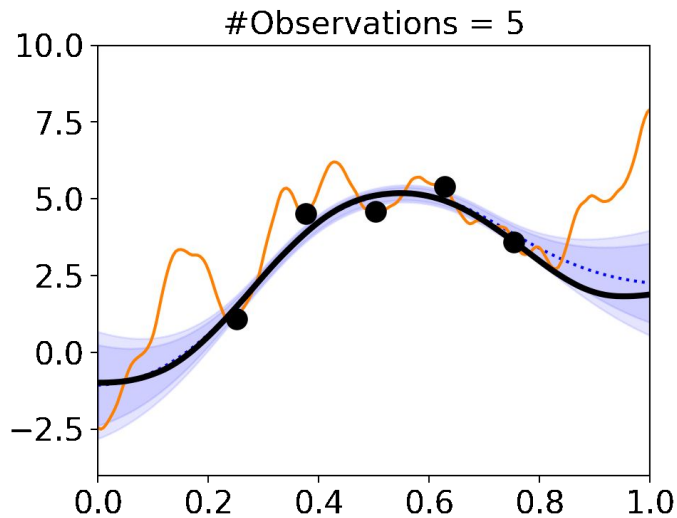
Over-exploring: searching where nothing is known.

- Recall $\mathbf{A}(\mathbf{x}) = \mu(\mathbf{x}) + b * \sigma(\mathbf{x})$
- Typically, $\sigma(\mathbf{x})$ increases away from support points.
- If $b \rightarrow \infty$, $\mathbf{A}(\mathbf{x})$ is typically maximal in between support points.
- If $b \rightarrow \infty$, $\mu(\mathbf{x})$ doesn't matter, so suggestions are spread uniformly, not focused on the better regions.
- In high dimensional spaces, uniform search is slow.



Focusing on the good regions means less attention to what the model thinks are bad regions.

- The model can be wrong.
 - Typically we use a Gaussian Process model for Bayesian Optimization, but the actual data is not generated by a Gaussian Process.
 - Or it's a needle-in-a-haystack problem and we haven't sampled the needle yet.



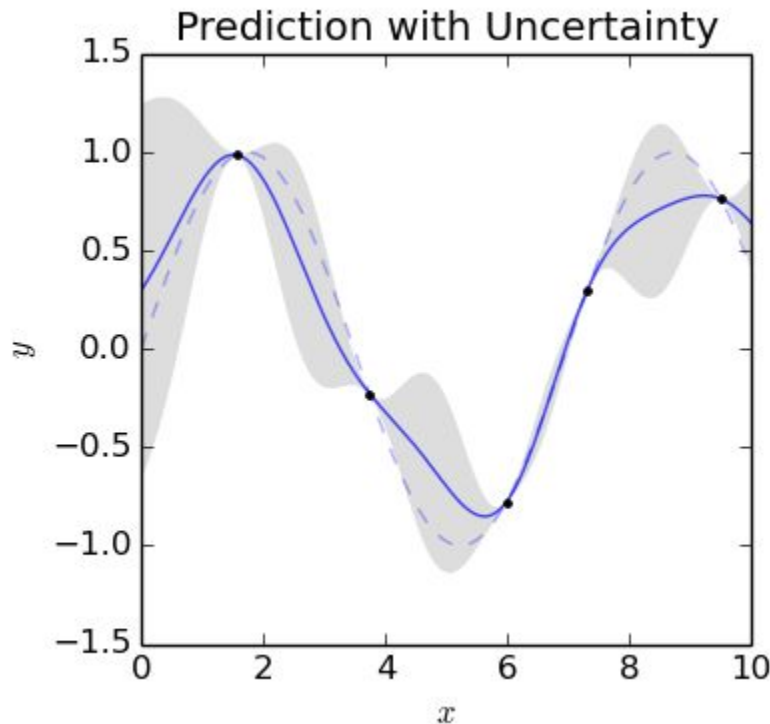
Unfocused Search takes Exponentially Long in High Dimensional Spaces.

- Imagine that to be close to the optimum of $F(\mathbf{x})$, you need to get D parameters all approximately correct.
- In this case, there are 3^D locations that need to be searched to find the optimum.



The Balance between Exploration and Exploitation is important.

- Recall $\mathbf{A}(x) = \mu(x) + b * \sigma(x)$
- Small b may get you trapped.
- Large b may be exponentially slow.
- There is theory, but the theory depends on the class of functions one is optimizing, and that's often not known.
- We mostly use $b=1.8$ (empirical), but spend some steps with $b \rightarrow \infty$ to force some exploration.



Overview

What is Black Box Optimization?

What is Bayesian Optimization?

How do You Make It Work?

Gaussian Process Regression.

Details Matter.

Features of Vizier

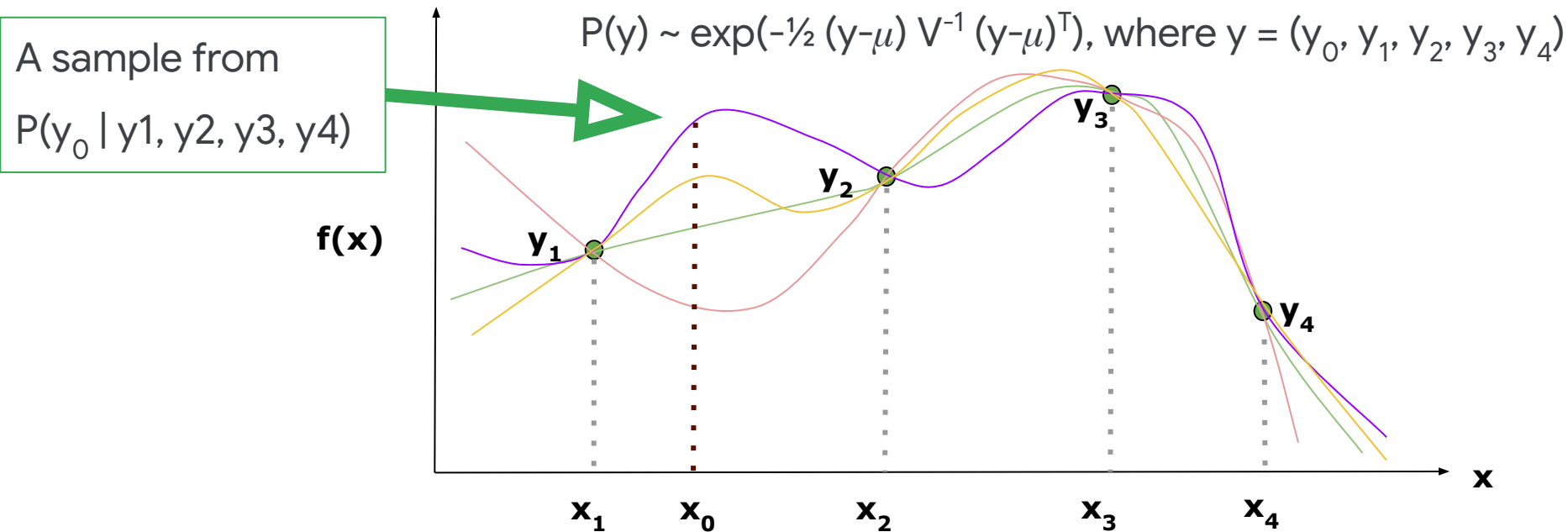
We've mentioned Gaussian Process Models...

- A Gaussian Process is a continuous generalization of a multivariate Normal distribution.
 - A multivariate normal distribution has a finite number of dimensions.
 - Measurements on the different dimensions are correlated
 - There is a positive-definite covariance matrix, V , that displays the correlations between different dimensions.

$$P(y) \sim \exp(-\frac{1}{2} (y-\mu)^T V^{-1} (y-\mu))$$

- Where x and μ are D -dimensional vectors and V is a $D \times D$ matrix.

Gaussian Process: Treating several points on the x-axis as a multivariate normal distribution.



Where $V_{i,j} = V(|x_i - x_j|)$. The covariance is a function of the distance between the x-values. Typically, $V_{i,j}$ is large when $|x_i - x_j|$ is small, and vice versa.

Overview

What is Black Box Optimization?

What is Bayesian Optimization?

How do You Make It Work?

Gaussian Process Regression.

Details Matter.

Features of Vizier

A Gaussian Process Typically has Hyperparameters.

Proprietary + Confidential

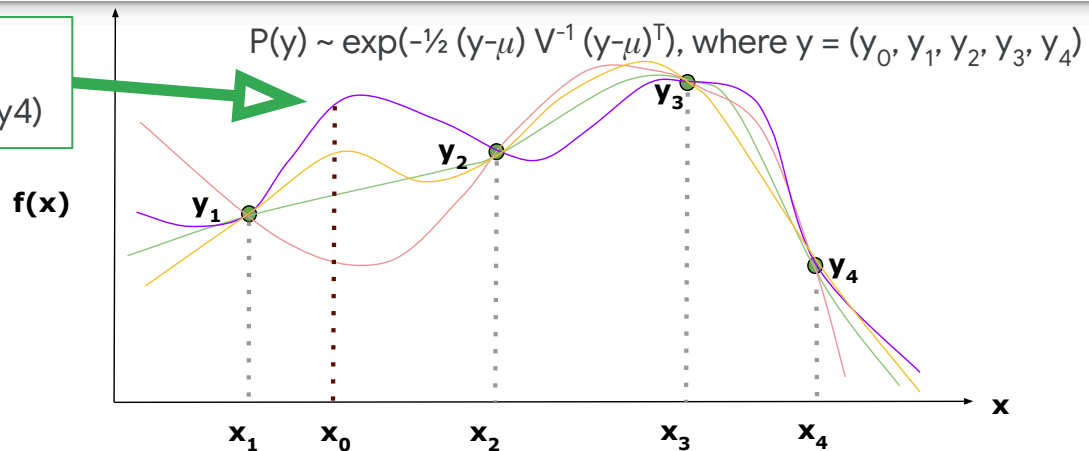
A sample from
 $P(y_0 | y_1, y_2, y_3, y_4)$

Measurement noise:
each sample is
independent &
uncorrelated.

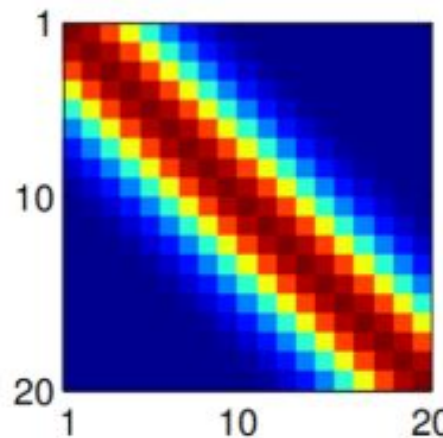
The width of
the kernel,
along the
x-axis.

Commonly $V_{i,j} = \delta_{i,j} \sigma_{\text{noise}}^2 + \sigma_{\text{signal}}^2 \kappa(|x_i - x_j| / \lambda)$

The magnitude of the
"signal", i.e. the smooth
Gaussian Process part.



$V_{i,j}$
=



(For evenly
spaced points
along a
1-dimensional
x-axis.)

A Gaussian Process Typically has Hyperparameters.

Proprietary + Confidential

Measurement noise:
each sample is
independent &
uncorrelated.

The width of
the kernel,
along the
x-axis.

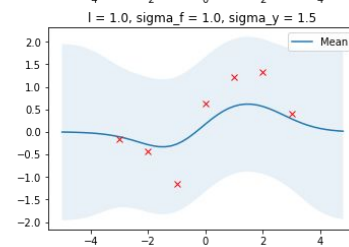
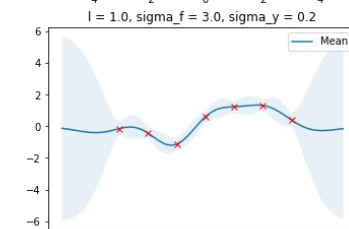
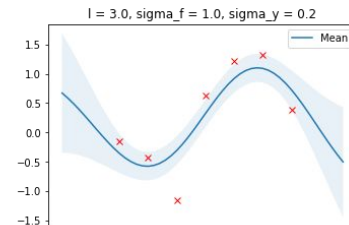
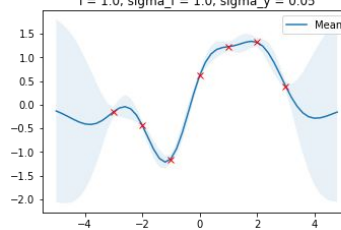
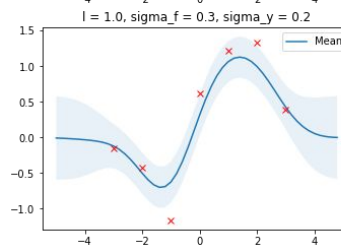
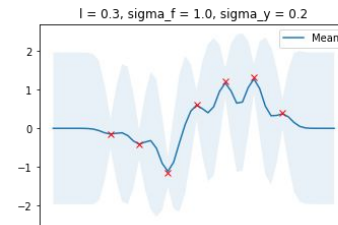
$$\text{Commonly } V_{i,j} = \delta_{i,j} \sigma_{\text{noise}}^2 + \sigma_{\text{signal}}^2 \kappa(|x_i - x_j| / \lambda)$$

The magnitude of the
"signal", i.e. the smooth
Gaussian Process part.

λ is
small

σ_{signal}^2
is
large

σ_{noise}^2
is
small



λ is
large

σ_{signal}^2
is
small

σ_{noise}^2
is
large

A Gaussian Process Typically has Hyperparameters.

$$P(y) \sim \exp(-\frac{1}{2} (y-\mu)^T V^{-1} (y-\mu))$$

Measurement noise:
each sample is
independent &
uncorrelated.

The width of
the kernel,
along the
x-axis.

Commonly $V_{i,j} = \delta_{i,j} \sigma_{\text{noise}}^2 + \sigma_{\text{signal}}^2 \kappa(|x_i - x_j| / \lambda)$

The magnitude of the
"signal", i.e. the smooth
Gaussian Process part.

- If there are hyperparameters, they generally need to be optimized.
- One maximizes $P(y)$ over the support points (training data).
- Occasionally, you'll see someone say that a "Gaussian Process doesn't depend on the y-values", and treat this as amazing.
 - Ignore them; they're confused.
 - The GP depends on your y-values via σ_{noise}^2 , σ_{signal}^2 and λ if you fit the GP hyperparameters to your data.
 - And if you don't fit the hyperparameters, the GP doesn't represent your data.

Bayesian Optimization (BO): GP-UCB

Assumptions for GP-UCB to work well:

- Functions are somewhat smooth on a continuous search space
- Homoscedastic noise
- Search space is not too large

OK, where are we?

- We have some system $F(x)$ and we want to find the x that maximizes $F(x)$.
- We have a Gaussian Process model $\mathbf{R}(x; \{D_i\})$.
 - We've tuned its hyperparameters σ^2_{noise} , σ^2_{signal} , and λ .
- We built an acquisition function $\mathbf{A}(\mathbf{R}(x; \{D_i\}))$
 - We've chosen a value of $\mathbf{b}$ (or an algorithm for choosing $\mathbf{b}$) that gives a good balance between exploration and exploitation.
- And now all we need to do -- to take one step of the Bayesian Optimization algorithm -- is to find the x that maximizes $\mathbf{A}(\mathbf{R}(x; \{D_i\}))$.
- *Whoops...*

- In one respect things are worse than when we started:
 - Even if $F(x)$ has a single maximum, $A(x; \{D\})$ can have more than one maximum per support point.
- But:
 - The cost of evaluating $A(x; \{D\})$ can be cheap compared to $F(x)$.
 - $A(x; \{D\})$ costs $O(N^2)$. (Where N is the number of support points.) So for modest N (e.g. $N < 1000$), we can afford lots of evaluations.
 - We don't need to accurately find $\operatorname{argmax}_x \{A(x; \{D\})\}$.
 - As long as we get close to a maximum, it'll still be a useful support point to improve $R(x; \{D\})$ and $A(x; \{D\})$.
 - We can seed the search for $\operatorname{argmax}_x \{A(x; \{D\})\}$ with the best known support points.

OK, where are we?

- We recognize that:
 - Bayesian Optimization is most valuable when $F(x)$ is expensive.
 - Bayesian Optimization is most valuable for problems with at most a few hundred parameters.
- We use a fast, cheap optimizer to find $\operatorname{argmax}_x \{A(x; \{D\})\}$
 - We use a variant of Firefly/Eagle.

Other Practical Details

- Parallel evaluation of $F(x)$
 - If $F(x)$ is really slow to evaluate, we might want to evaluate in parallel to speed the overall optimization.
- Label (y -value) processing.
 - Vizard's behavior is designed to be invariant under affine transformation of y .
 - Vizard maps labels into a Normal distribution to be robust against arbitrarily bad values of $F(x)$.
 - They happen more often than you might expect...

Overview

What is Black Box Optimization?

What is Bayesian Optimization?

How do You Make It Work?

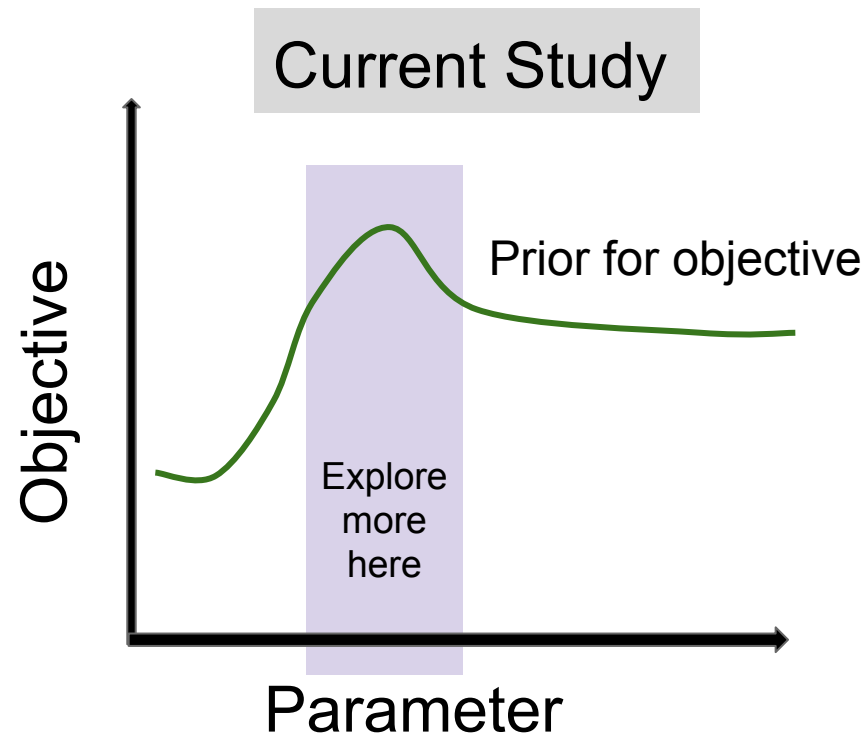
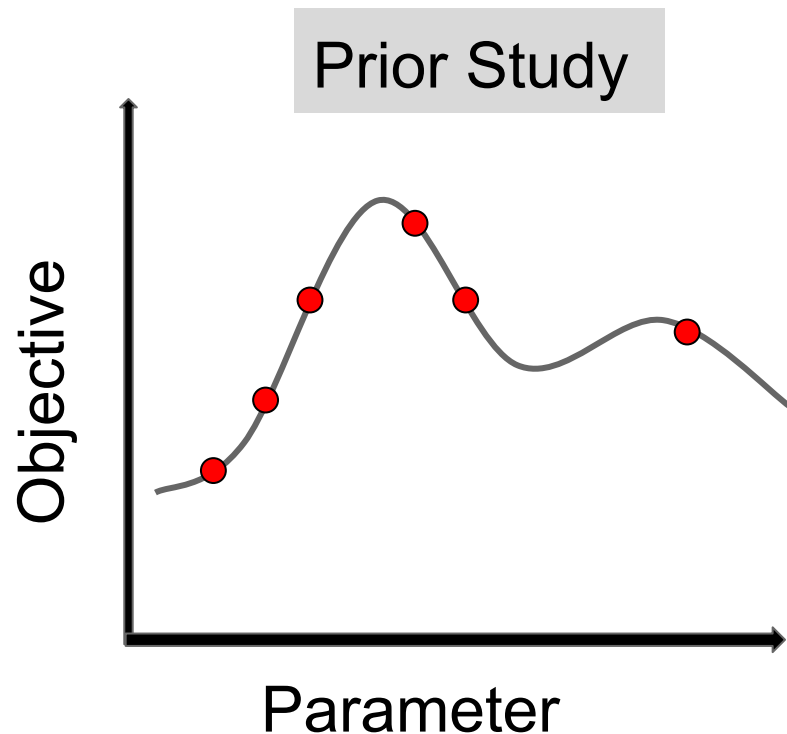
Gaussian Process Regression.

Details Matter.

Features of Vizier

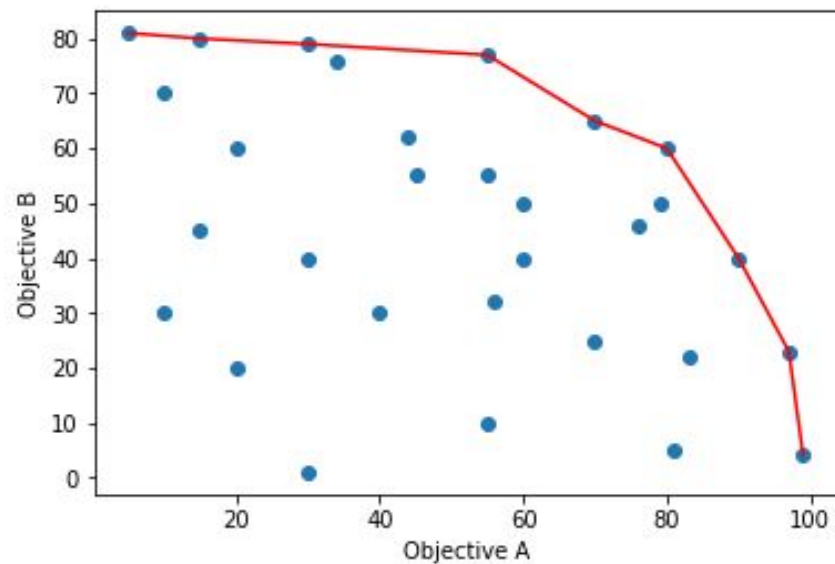
Transfer Learning

- Can be very useful for sequences of similar problems (e.g. this week's data).



Pareto Optimization Using Vizier

- Iterative search for the dominating points:



Conditional Parameters

- Tree-structured parameters. Maybe you're choosing between two systems.
 - System A has parameters AA, AB, and AC
 - System B has parameters BA and BB
 - Different ranges and meanings for each.

Contextual Optimization

- Learn a function rather than a optimal vector.
 - Learn a vector of parameters for a given context.
 - E.g. learn the optimal dose of a drug as a function of the patient's weight and sex.

Constrained Optimization

- Optimize Metric A under the constraint that Metric B must be positive.
 - E.g. In the context of wind-power kites ([Google Project Makhani](#)).
 - Fly the kite to collect maximum power, under the constraint:
 - Don't get too close to the ground.
 - This uses a second GP Regressor to estimate the value of the constraint.

Conditional Parameters

- Tree-structured parameters. Maybe you're choosing between two systems.
 - System A has parameters AA, AB, and AC
 - System B has parameters BA and BB
 - Different ranges and meanings for each.

Contextual Optimization

- Learn a function rather than a optimal vector.
 - Learn a vector of parameters for a given context.
 - E.g. learn the optimal dose of a drug as a function of the patient's weight and sex.



Thank you.

Papers about Vizier

- [Google Vizier: A Service for Black-Box Optimization](#) (2017)
- [Bayesian Optimization for a Better Dessert](#) (2017)
- [Open Source Vizier: Distributed Infrastructure and API for Reliable and Flexible Blackbox Optimization.](#) (2022)
- [The Vizier Gaussian Process Bandit Algorithm](#) (2024)

Open Source Vizier

- <https://oss-vizier.readthedocs.io/en/latest/guides/developer/designers.html>
- https://github.com/google/vizier/blob/main/vizier/_src/algorithms/designers/gp_bandit.py
- [Github project page.](#)

Vertex Vizier released in 2020

<https://cloud.google.com/vertex-ai/docs/vizier/overview>

Vertex AI > Documentation > Guides

Was this helpful?  

Vertex AI Vizier overview



[Send feedback](#)

Vertex AI Vizier is a black-box optimization service that helps tune hyperparameters in complex machine learning (ML) models. When ML models have many different hyperparameters, it can be difficult and time consuming to tune them manually. Vertex AI Vizier optimizes your model's output by tuning the hyperparameters for you.

Black-box optimization is the optimization of a system that meets either of the following criteria:

- Doesn't have a known [objective function](#)  to evaluate.
- Is too costly to evaluate by using the objective function, usually due to the complexity of the system.

Additional Vertex AI Vizier functionality

Vertex AI Vizier optimizes hyperparameters of ML models, but it can also perform other optimization tasks.

OSS Vizier released in 2022

<https://github.com/google/vizier>

Open Source Vizier: Reliable and Flexible Black-Box Optimization.



pypi package 0.1.13 ci passing docs_test passing

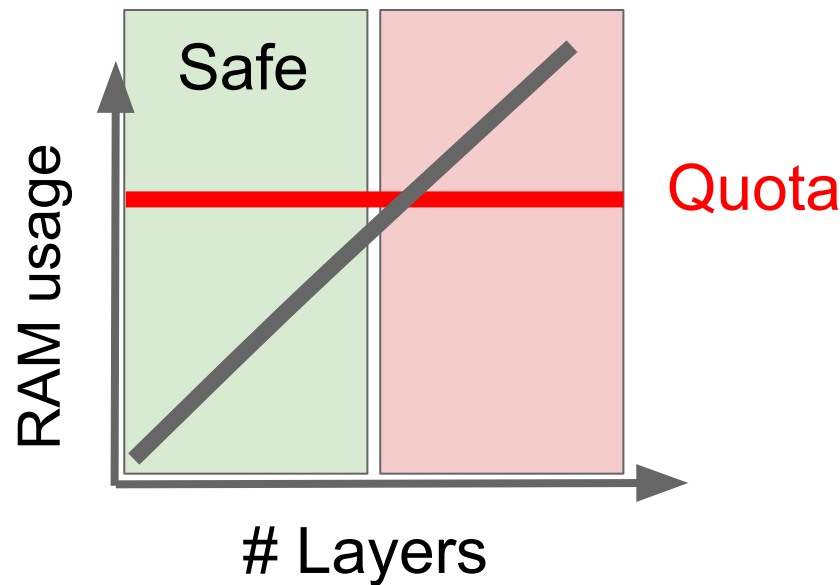
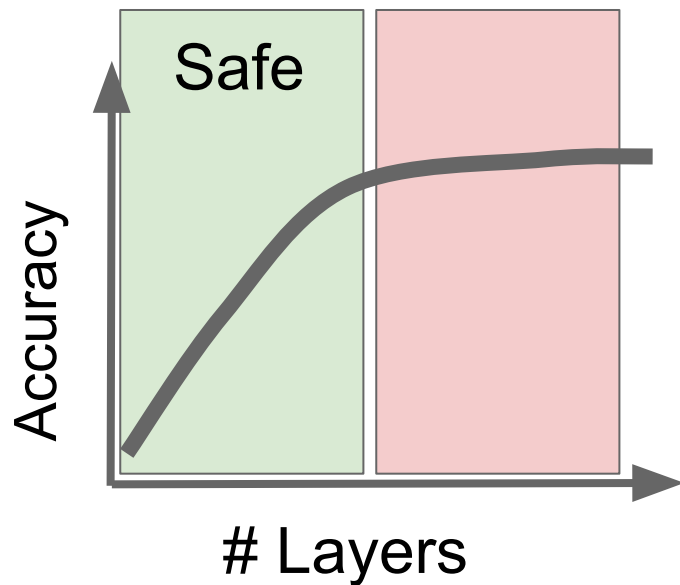
[Google AI Blog](#) | [Getting Started](#) | [Documentation](#) | [Installation](#) | [Citing and Highlights](#)

What is Open Source (OSS) Vizier?

[OSS Vizier](#) is a Python-based service for black-box optimization and research, based on [Google Vizier](#), one of the first hyperparameter tuning services designed to work at scale.

AutoML Using Vizier

- Constrained exploration also helps in AutoML use cases
- Optimize objective without violating (initially unknown) safety constraints

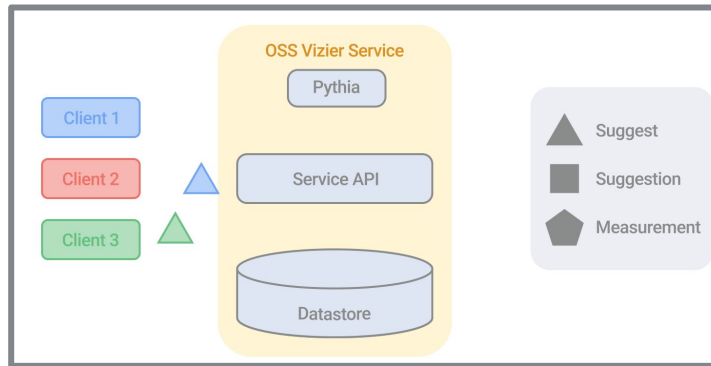


Types of Optimization Packages

- **Services:** Host algorithms & store data on a server.
 - Flexible choice of programming language, most scalable, supports distributed use cases
 - Write client-side code as if any RPC can fail.
- **Frameworks:** Encapsulate entire optimization (both algorithm + client code)
 - Convenient, full automation
 - Algorithm has to be the same programming language as client, entire eval pipeline has to be known ahead of time
- **Libraries:** Implement algorithms
 - Easiest to start with
 - Doesn't support distributed processing, not scalable, algorithm has to be the same programming language as the client
-

Vizier's Architecture

- Vizier is a service, because it has to support very different workflows:
 - The time it takes a client to evaluate a suggestion is arbitrary
 - Can be seconds or weeks
 - Clients can be asynchronous or Synchronous (Batched)
 - Clients can permanently fail or retry



Vizier's Architecture

- Vizier uses RPCs (remote procedure calls):

vizier / vizier / _src / service / vizier_service.proto

Code Blame 454 lines (396 loc) · 15.5 KB

```
15 // An Open-Source port of the Vizier API, based on Cloud AI Platform Vizier API.
16 service VizierService {
17     // Creates a Study. A resource name will be generated after creation of the
18     // Study.
19     rpc CreateStudy(CreateStudyRequest) returns (Study) {
20         option (google.api.http) = {
21             post: "{parent=owners/*/studies}"
22             body: "study"
23         };
24         option (google.api.method_signature) = "parent,study";
25     }
26 }
```



A high performance, open source universal RPC framework

Vizier's Architecture

- Simplified view:

